DAMANHOUR UNIVERSITY

Faculty of Computers and Information

GRADUATION PROJECT

MedLink
YOUR MEDICINE, OUR LINK

MedLink: Smart Pharmacy Platform

Next-Generation Digital Pharmacy Management & Healthcare Delivery

PREPARED BY

Omar Kareem Mohamed

Ahmed Metwally Yehia

Mostafa Mohamed Saad

Rahma Riad Mohamed

Esraa Saad Ahmed

SUPERVISED BY

Dr. Shaimaa Sami

ASSISTANT / CO-SUPERVISOR

Eng. Mahmoud

Academic Year: 2026 – 2027

Abstract

MedLink is a smart pharmacy platform designed to make finding, ordering, and receiving medicines and healthcare products easier for patients. Patients may face difficulties when a required medicine is not available at the pharmacy they visit. They may need to contact or visit several pharmacies to find the required medicine, which takes time and effort. This can be more difficult when the patient has a prescription that contains several medicines or when the required medicines are available at different pharmacies. In addition, the process of finding medicines, contacting pharmacists, placing orders, making payments, and arranging delivery may be handled through separate services, making the overall experience less simple and less connected.

MedLink aims to bring these services together in one connected platform. Patients can create an account, upload a prescription, and use an AI-based system to read the prescription and identify the required medicines. After identifying the medicines, the system can search connected pharmacies and check their availability. This helps patients know where the required medicines can be found before placing an order or visiting a pharmacy. Patients can also search for medicines and healthcare products, view available products, communicate with pharmacists, place and manage orders, make payments, and receive their orders through delivery services. On the pharmacy side, the platform provides tools for managing medicines, inventory, products, and incoming orders. This creates a more organized connection between patients and participating pharmacies.

From a software engineering perspective, MedLink will include web and mobile applications, a backend system, a database, and AI techniques for prescription understanding. The system will be designed to provide a simple and clear user experience while supporting the main pharmacy and ordering processes. The AI component will assist in reading prescriptions and identifying medicines, but it will not replace pharmacists or doctors in making medical decisions. The project will also focus on connecting pharmacies with delivery services to support the order fulfillment process. The expected outcome is a connected pharmacy platform that reduces the effort required to find medicines, makes ordering more convenient, improves pharmacy inventory and order management, and provides patients with an easier way to access medicines and healthcare products.

Keywords: Smart Pharmacy, Online Pharmacy, Prescription Understanding, Medicine Availability, Pharmacy Management, Artificial Intelligence (AI).

Table of Contents

1 Introduction 1

1.1 Problem Overview 1

1.2 Motivation 1

1.3 Problem Statement 1

1.4 Goals / Objectives 2

1.5 Software Development Methodology 2

2 Literature Review 3

2.1 Related Work / Existing Solutions 3

2.2 Feature Matrix 3

2.3 Benchmarking 3

2.4 Research Gap 3

2.5 Scenarios 4

3 Requirement Engineering 5

3.1 Surveys / Questionnaires / Interviews 5

3.2 Functional Requirements 5

3.3 Non-Functional Requirements 5

3.4 Use Case Diagram 6

3.5 Use Case Tables 6

4 Design 8

4.1 Introduction 8

4.2 Sequence Diagrams 8

4.3 Class Diagram 8

4.4 ERD 9

4.5 Database Schema 9

4.6 Data Dictionary 9

4.7 DFD Level 0 & 1 9

4.8 Algorithms / Pseudocode 10

4.9 Interface Design 10

5 Implementation Aspects 11

5.1 Overall System Architecture 11

5.2 Tools & Technologies 11

6 Testing & Evaluation 13

6.1 Introduction 13

6.2 Testing Strategy . . . 13
6.3 Unit Testing . . . 13
6.4 Integration Testing . . . 13
6.5 System / E2E Testing . . . 13
6.6 Test Cases . . . 13
6.7 Performance Testing . . . 14
6.8 Security Testing . . . 14
6.9 Usability Testing . . . 14
6.10 Testing Environment . . . 14
6.11 Issues & Fixes . . . 14
6.12 Testing Summary . . . 14
6.13 Conclusion . . . 14

7 Testing and Evaluation . . . 15
7.1 Introduction . . . 15
7.2 Testing Strategy and Methodology . . . 15
7.3 Test Scenarios and Test Cases . . . 15
7.4 Non-Functional Testing . . . 16
7.5 Requirements Traceability Matrix . . . 16
7.6 Evaluation and Discussion . . . 16
7.7 Summary . . . 16

8 Future Work & Conclusion . . . 17
8.1 Future Work . . . 17
8.2 Conclusion . . . 17

Bibliography . . . 17

Appendices . . . 19

A Git Repository . . . 19

List of Tables

2.1 Feature Comparison Matrix 3

3.1 Use Case UC01: Process Electronic Prescription 7

4.1 Data Dictionary: Medications Table 10

6.1 Core System Test Cases 14

7.1 Test Cases for Scenario 1 15
7.2 Test Cases for Scenario 2 16
7.3 Requirements Traceability Matrix 16

List of Figures

3.1 MedLink System Use Case Diagram 6

4.1 Sequence Diagram for Prescription Validation Workflow 8
4.2 MedLink Core Class Diagram 9
4.3 MedLink Entity Relationship Diagram (ERD) 9

A.1 Git Insights 19

CHAPTER 1

Introduction

1.1 Problem Overview

Finding medicines can be difficult when a required medicine is not available at the pharmacy a patient visits. In this situation, the patient may need to contact or visit several pharmacies to find the required medicine. This can take a lot of time and effort, especially when the patient needs the medicine quickly or when the prescription contains several medicines.

The problem becomes more challenging when the required medicines are available at different pharmacies. Patients may not have an easy way to know which pharmacies have the medicines in stock before visiting them. As a result, they may spend additional time searching for medicines that may still not be available.

In addition, other parts of the medicine ordering process, such as prescription handling, communication with pharmacists, placing orders, payment, and delivery, may be managed through separate services. This makes the overall process less connected and can create unnecessary steps for both patients and pharmacies.

1.2 Motivation

The main motivation behind MedLink is to make the process of finding, ordering, and receiving medicines easier and more convenient for patients. The project aims to provide a connected platform where patients can access the main services they need without depending on several separate services.

Another motivation is to help patients find medicine availability before visiting a pharmacy. By connecting multiple pharmacies through the platform, patients can search for required medicines and check their availability. This can reduce unnecessary visits and the time spent contacting different pharmacies.

MedLink is also motivated by the need to improve pharmacy operations. Pharmacies can use the platform to manage medicines, inventory, products, and incoming orders in a more organized way. Connecting pharmacies with patients, pharmacists, and delivery services can create a smoother process for handling and completing medicine orders.

1.3 Problem Statement

Patients may face difficulties when they need a medicine that is not available at the pharmacy they visit. They may have to contact or visit several pharmacies before finding the required medicine. This problem becomes more difficult when a prescription contains multiple medicines that are not available at the same pharmacy, as the patient may need to search for each medicine separately.

The process of obtaining medicines can also involve several other steps. Patients may need to handle their prescription, ask pharmacists about medicines, check availability, place an order, make a payment, and arrange delivery. When these activities are handled through separate services, the patient experience becomes less simple and more time-consuming.

MedLink addresses these problems by providing one connected pharmacy platform that brings the main parts of the process together. Patients can upload prescriptions, use AI to assist in identifying the required medicines, check their availability in connected pharmacies, communicate with pharmacists, place and manage orders, and receive their orders through delivery services. The platform also supports pharmacies in managing their medicines, inventory, and orders.

1.4 Goals / Objectives

- **Primary Goal**: Develop a connected smart pharmacy platform that makes finding, ordering, and receiving medicines and healthcare products easier for patients while helping pharmacies manage their medicines, inventory, and orders.
- **Key Objectives**:
  - Connect patients with pharmacies, pharmacists, and delivery services through one platform.
  - Allow patients to upload prescriptions and use an AI-based system to assist in identifying the required medicines.
  - Enable patients to search connected pharmacies and check the availability of required medicines.
  - Support medicine and healthcare product search, ordering, payment, and delivery.
  - Provide pharmacy tools for managing medicines, inventory, products, and incoming orders.
  - Support communication between patients and pharmacists through the platform.

1.5 Software Development Methodology

MedLink follows an Agile Scrum methodology to organize the project work and develop the system requirements and design in an iterative way. This methodology allows the team to divide the project into smaller tasks and work on each part in an organized and manageable way.

The team uses short work cycles to discuss the project requirements, review the current work, and identify any changes that may be needed. This helps the team keep the project requirements clear and allows improvements to be made during the development of the project documentation and system design.

The Agile approach is suitable for MedLink because the project includes several connected parts, such as patient services, pharmacy management, prescription understanding, medicine availability, ordering, and delivery. Working on these parts step by step helps the team understand their relationships and build a clear overall system design.

During this phase, the methodology is mainly used to organize requirements analysis, literature review, system modeling, database design, architecture design, and interface planning. Regular reviews are used to make sure that the different parts of the project remain consistent with the main project goals.

CHAPTER 2

Literature Review

Explaining what your project does that is new or is better than existing work in the same field, reviewing relevant literature and existing commercial and open-source pharmacy solutions.

2.1 Related Work / Existing Solutions

Review relevant published literature, industry platforms, and existing commercial or open-source pharmacy management systems. Discuss their architectural strengths and practical limitations. Test citation [1].

2.1.1 Existing System A: Traditional Enterprise Pharmacy Systems

Description of established on-premise systems, highlighting their inventory features, high licensing costs, lack of real-time patient connectivity, and rigid workflows.

2.1.2 Existing System B: E-Prescription & Pharmacy Apps

Analysis of modern mobile-first applications, focusing on prescription upload convenience while highlighting gaps in inventory synchronization and comprehensive clinical checks.

2.2 Feature Matrix

Provide a comprehensive comparison matrix contrasting core capabilities across existing solutions and the proposed MedLink platform.

Table 2.1: Feature Comparison Matrix

Feature / Capability	System A	System B	System C	MedLink Platform
Real-Time Inventory Sync	✓	×	×	✓
Electronic Prescription Handling	×	✓	×	✓
Automated Drug Interaction Checking	×	×	✓	✓
Patient Direct Ordering & Tracking	×	✓	×	✓
Predictive Stock Replenishment Alerts	×	×	×	✓

2.3 Benchmarking

Evaluate the quantitative and qualitative performance criteria of current solutions, including response times, reliability, accessibility, operational cost, and overall usability.

2.4 Research Gap

Highlight the shortcomings of current market solutions and define the exact gap that the MedLink platform bridges, particularly the integration gap between patient convenience, clinical decision support, and automated inventory logistics.

2.5 Scenarios

Illustrate standard real-world usage scenarios demonstrating how the proposed system addresses the identified gaps:

- **Scenario 1: Automated Prescription Processing**: A patient uploads a digital prescription; the pharmacist receives real-time validation, automatic dosage check, and stock reservation.
- **Scenario 2: Critical Drug-Drug Interaction Prevention**: The system automatically flags a potential adverse interaction between two prescribed medications, warning the pharmacist before dispensing.
- **Scenario 3: Low-Stock Predictive Restocking**: The inventory engine detects approaching depletion for seasonal medications and prompts the inventory manager with automated restock proposals.

CHAPTER 3

Requirement Engineering

This chapter details the requirement engineering process for the MedLink platform, capturing field findings, stakeholder expectations, and formal functional and non-functional specifications.

3.1 Surveys / Questionnaires / Interviews

Detail the user research conducted with pharmacists, pharmacy technicians, physicians, and patients to discover common operational bottlenecks and pain points.

3.1.1 Stakeholder Analysis

Identify and profile the key stakeholders: Pharmacists, Pharmacy Managers, Patients, System Administrators, and Healthcare Regulators.

3.1.2 Survey Methodology and Key Findings

Summary of the structured questionnaire and interviews conducted, statistical analysis of user responses, and the derived feature priorities.

3.2 Functional Requirements

Specify the comprehensive functional requirements of the system. A good requirement states something that is **necessary**, **verifiable**, and **attainable** [2].

3.2.1 Core Functional Requirements List

1. **FR01 — User Authentication & Role-Based Access Control**: The system shall authenticate users and enforce distinct permissions for Pharmacists, Patients, and Administrators.
2. **FR02 — Prescription Management**: The system shall enable patients to submit electronic prescriptions and allow licensed pharmacists to review, validate, and dispense them.
3. **FR03 — Real-Time Inventory Control**: The system shall automatically update inventory levels upon dispensing and maintain batch numbers, expiration dates, and unit prices.
4. **FR04 — Automated Drug Interaction Checking**: The system shall evaluate medication combinations against clinical databases and trigger safety alerts when conflicts occur.
5. **FR05 — Order Processing & Notification**: The system shall track order statuses and send automated notification updates to patients.

3.3 Non-Functional Requirements

Specify the non-functional quality attributes governing system performance, security, and reliability:

- **Performance**: The system shall return search query results and prescription validation feedback within 1.5 seconds under standard load.

- **Security**: All sensitive patient data, authentication credentials, and clinical records shall be encrypted at rest (AES-256) and in transit (TLS 1.3).
- **Availability & Reliability**: The system shall maintain an uptime of 99.5% during core operational hours with automated backup routines.
- **Usability**: The user interface shall adhere to responsive design principles, ensuring accessibility across mobile and desktop devices.
- **Scalability**: The architecture shall support concurrent sessions across multiple pharmacy branches without performance degradation.

3.4 Use Case Diagram

Provide the comprehensive UML Use Case diagrams modeling system actors and their primary interactions with the MedLink platform.

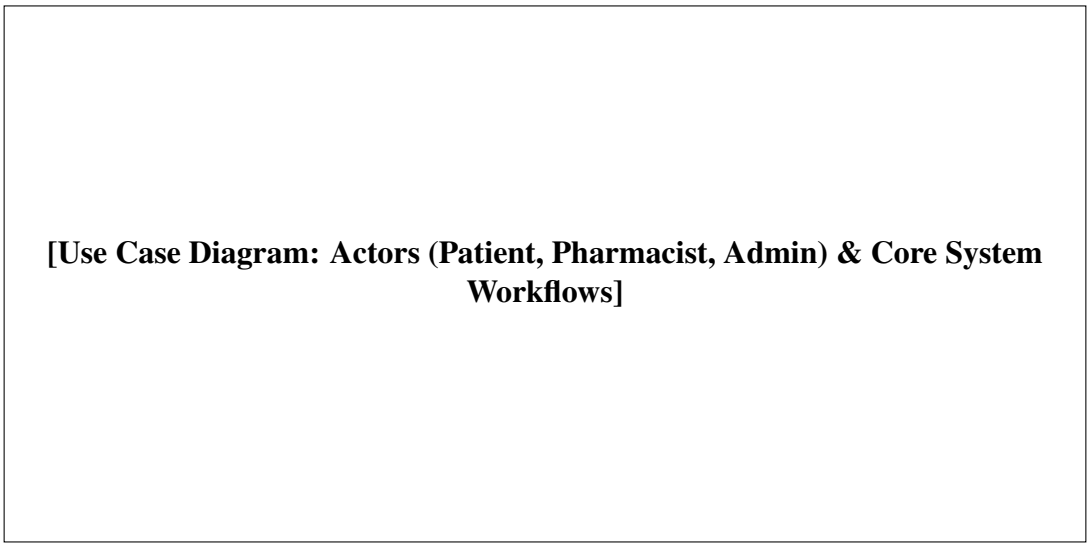


Figure 3.1: MedLink System Use Case Diagram

3.5 Use Case Tables

Provide structured use case specifications for the primary workflows of the MedLink platform.

Table 3.1: Use Case UC01: Process Electronic Prescription

Use Case ID	UC01
Use Case Name	Process Electronic Prescription
Primary Actor	Pharmacist
Pre-condition	Pharmacist is authenticated and a pending prescription exists in the queue.
Trigger	Pharmacist opens pending prescription queue.
Main Scenario	1. Pharmacist selects pending prescription. 2. System displays medication items, prescribed dosages, and patient profile. 3. System automatically runs clinical drug-interaction checks. 4. System verifies current stock availability. 5. Pharmacist approves and validates order for preparation. 6. System reserves inventory items and notifies patient.
Post-condition	Prescription status updated to 'Validated'; stock reserved.
Exceptions	Insufficient stock triggers partial-fulfillment option or restocking request.

CHAPTER 4

Design

Containing a comprehensive description of the architectural models, behavioral interactions, data models, and interface specifications chosen for the MedLink platform.

4.1 Introduction

Introductory overview of the system architecture, design principles, and patterns adopted to fulfill the requirements specified in Chapter 3.

4.2 Sequence Diagrams

UML Sequence diagrams describing object interactions and asynchronous message exchanges for the critical business scenarios of the MedLink platform.

4.2.1 Sequence Diagram: Prescription Validation Workflow

Detailed interaction flow between the Patient client, API Gateway, Prescription Controller, Drug Interaction Service, and Database repository.

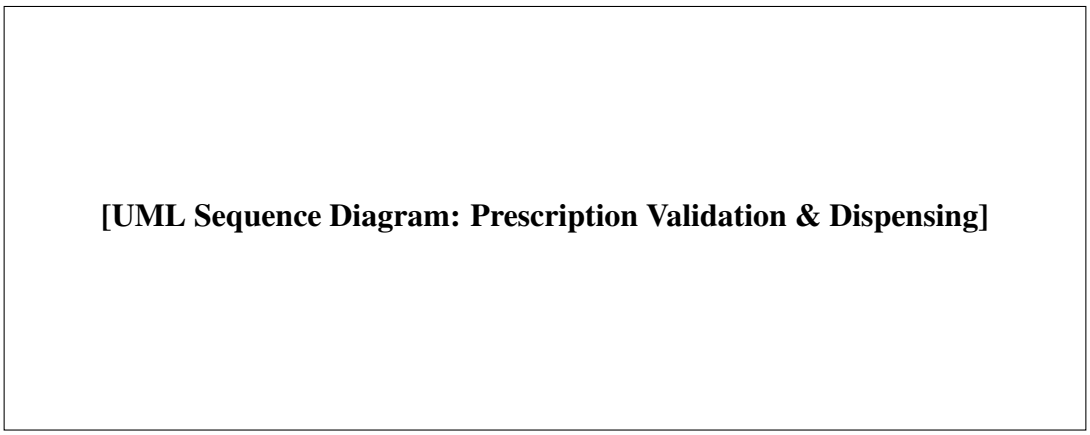


Figure 4.1: Sequence Diagram for Prescription Validation Workflow

4.2.2 Sequence Diagram: Inventory Order and Stock Update

Interaction flow between Pharmacist interface, Inventory Management Service, and Notification Service.

4.3 Class Diagram

Provide the domain model and static class structure representing core entities, services, controllers, and their object-oriented relationships.

[UML Class Diagram: Core Domain Entities, Services, and Repositories]

Figure 4.2: MedLink Core Class Diagram

4.4 ERD

The Entity Relationship Diagram (ERD) defining system data entities, their attributes, primary keys, and cardinality relationships across the platform.

[Entity Relationship Diagram (ERD): Users, Prescriptions, Medications, Orders]

Figure 4.3: MedLink Entity Relationship Diagram (ERD)

4.5 Database Schema

Formal relational database schema defining table structures, column data types, constraints, foreign key cascades, and indexing strategies.

4.6 Data Dictionary

Provide a comprehensive data dictionary specifying table names, field descriptions, data types, constraints, and defaults.

4.7 DFD Level 0 & 1

Data Flow Diagrams modeling information movement, transformations, data stores, and external entities.

4.7.1 DFD Level 0 (Context Diagram)

High-level context diagram representing the MedLink system as a single central process interacting with external actors (Patients, Pharmacists, Suppliers, Payment Gateways).

Table 4.1: Data Dictionary: Medications Table

Field Name	Data Type	Constraint	Description
medication_id	UUID	PK, NOT NULL	Unique identifier for medication item
trade_name	VARCHAR(150)	NOT NULL	Commercial brand name of the drug
generic_name	VARCHAR(150)	NOT NULL	Active pharmaceutical substance
dosage_form	VARCHAR(50)	NOT NULL	Tablet, capsule, syrup, or injection
unit_price	DECIMAL(10,2)	NOT NULL	Retail price per packaging unit
stock_quantity	INT	DEFAULT 0	Current available quantity in inventory
expiration_date	DATE	NOT NULL	Batch expiration date

4.7.2 DFD Level 1 (Decomposition Diagram)

Decomposed data flow diagram detailing sub-processes: Authentication, Prescription Processing, Clinical Interaction Verification, Inventory Logistics, and Reporting.

4.8 Algorithms / Pseudocode

Detailed pseudocode and algorithmic logic for critical computational tasks in the MedLink platform, such as automated drug-drug conflict detection and predictive inventory reorder points.

Algorithm: Automated Drug-Drug Conflict Detection

Input: Array of prescribed medications $M = [m_1, m_2, \ldots, m_n]$, Patient profile P
Output: Array of safety alerts $Alerts$

1. Initialize $Alerts \leftarrow \emptyset$
2. For each pair (m_i, m_j) in M where $i < j$:
   (a) Query clinical interaction matrix for interaction $(m_i.generic_name, m_j.generic_name)$
   (b) If conflict detected:
      i. Append alert with severity level (Mild, Moderate, Severe) to $Alerts$
3. Check medication contraindications against patient known allergies in P
4. Return $Alerts$

4.9 Interface Design

Detailed user interface design wireframes, responsive UI architecture, navigation flow, and screen mockups for both Pharmacist portal and Patient mobile views.

CHAPTER 5

Implementation Aspects

Containing a comprehensive overview of the overall system architecture, technology stack, development toolchains, and deployment environments selected for the MedLink platform.

5.1 Overall System Architecture

Detailed discussion of the multi-tier client-server architecture, API Gateway routing, microservices/modular backend components, and cloud infrastructure integration.

5.1.1 Architectural Pattern and Tiers

Description of the 3-Tier / Layered architecture separating the Presentation Layer (Web/Mobile clients), Business Logic Layer (REST API services, business engines), and Data Persistence Layer (Relational databases and cache stores).

5.1.2 Cloud and Deployment Architecture

Overview of the hosting infrastructure, containerization (Docker), continuous integration / continuous deployment (CI/CD) pipelines, and reverse proxy configurations (Nginx).

5.2 Tools & Technologies

Detailed review of the programming languages, frameworks, databases, libraries, and developer tools employed throughout the MedLink implementation.

5.2.1 Frontend Technologies

- **Framework**: Modern responsive framework (React / Next.js / Flutter).
- **Styling & UI Components**: Tailored responsive design system, component libraries, and interactive visualization charts.
- **State Management**: Centralized client-side state handling and asynchronous API data fetching.

5.2.2 Backend Technologies

- **Runtime & Framework**: High-performance server environment (Node.js / Express or Python / FastAPI / Django).
- **API Architecture**: RESTful API standards with secure JSON Web Token (JWT) authorization headers.

5.2.3 Database and Caching Layer

- **Primary Database**: PostgreSQL / MySQL for relational integrity, transactional ACID safety, and complex querying.

- **Caching & Session Store**: Redis for caching frequent drug search results and maintaining session states.

5.2.4 Development Environment and Toolchain

- **IDEs**: Visual Studio Code / JetBrains IDEs with specialized linter and formatting configurations.
- **Version Control**: Git and GitHub for collaborative branching, pull requests, and automated testing workflows.
- **API Testing**: Postman / Swagger for endpoint documentation and automated contract testing.

CHAPTER 6

Testing & Evaluation

Containing a comprehensive testing plan, executed verification scenarios, quality metrics, performance and security assessments, and evaluation of results against the defined goals.

6.1 Introduction

Introductory remarks detailing the quality assurance objectives, testing standards, and verification methodologies applied across the MedLink platform.

6.2 Testing Strategy

Overview of the testing pyramid and multi-tier quality assurance lifecycle, establishing the scope and criteria for unit, integration, system, and user evaluation.

6.3 Unit Testing

Verification of individual isolated components, business logic functions, API controllers, and algorithmic helper methods.

6.3.1 Backend Unit Tests

Testing authentication services, order calculation logic, and interaction conflict evaluation algorithms using automated unit test runners.

6.3.2 Frontend Component Tests

Testing UI components, form validation rules, and client-side state transitions in isolation.

6.4 Integration Testing

Testing inter-module communication, database transactions, API endpoint contracts, and external notification gateways.

6.5 System / E2E Testing

End-to-end testing validating comprehensive user journeys from prescription submission to automated inventory deduction and dispensing.

6.6 Test Cases

Structured test cases covering functional requirements and boundary conditions.

Table 6.1: Core System Test Cases

Test ID	Test Case Description	Req Code	Input Data	Expected Result
TC01	Authenticate pharmacist with valid credentials	FR01	Valid email and password	Successful login; JWT issued; redirect to portal
TC02	Upload and validate electronic prescription	FR02	Prescription image / digital file	Prescription stored with status 'Pending'
TC03	Detect conflicting drug interaction	FR04	Combination of interacting drugs	High-priority clinical interaction alert displayed
TC04	Automatic inventory deduction upon dispensing	FR03	Validated order items	Stock quantity decremented; transaction logged

6.7 Performance Testing

Benchmarking system latency, API response times, database query execution times, and throughput under simulated concurrent user traffic.

6.8 Security Testing

Evaluating system resilience against security vulnerabilities, including OWASP Top 10 risks (SQL Injection, Cross-Site Scripting, Broken Access Control), token expiration, and data encryption.

6.9 Usability Testing

Evaluation of user satisfaction, task completion rate, and ease of navigation conducted with representative pharmacist and patient participant groups.

6.10 Testing Environment

Specification of hardware specifications, operating systems, browser matrices, mobile emulators, database staging instances, and automated test runners.

6.11 Issues & Fixes

Documentation of critical bugs, race conditions, or usability bottlenecks identified during test execution, alongside the technical fixes implemented.

6.12 Testing Summary

Quantitative summary of test execution metrics, including test pass rates, code coverage percentages, defect density, and verification sign-offs.

6.13 Conclusion

Concluding evaluation demonstrating that the system successfully satisfies all verification criteria and fulfills the operational goals established in Chapter 1.

CHAPTER 7

Testing and Evaluation

Containing a comprehensive testing plan, executed test scenarios, requirements traceability, and evaluation of results against the defined objectives.

7.1 Introduction

Introductory remarks detailing the testing methodology, objectives of the testing phase, and quality assurance goals.

7.2 Testing Strategy and Methodology

Describe the overall test strategy applied to the project.

- **Unit Testing**: Testing individual components and functions in isolation.
- **Integration Testing**: Verifying communication between subsystems and database/API layers.
- **System Testing**: End-to-end evaluation of the integrated software.
- **User Acceptance Testing (UAT)**: Validating system workflows against real user expectations.

7.3 Test Scenarios and Test Cases

7.3.1 Test Scenario 1

Describe possible users' actions and behaviors. Determine the technical aspects of the requirement. Ascertain possible scenarios of system failure. You may refer to use cases in SRS (Chapter 3) and sequence diagrams in SDD (Chapter 4).

Test Cases for Scenario 1

Test cases for the scenario mentioned in Section 7.3.1 are shown in Table 7.1.

Table 7.1: Test Cases for Scenario 1

Test Case ID	Test Case Desc	Req Code	Test Data	Expected Result
TC01	Verify user login with valid credentials	FR01	Valid email and password	Successful authentication and redirect
TC02	Verify user login with invalid password	FR01	Valid email, wrong password	Display error message and deny access

7.3.2 Test Scenario 2

Describe possible users' actions and behaviors for another critical workflow.

Test Cases for Scenario 2

Test cases for the scenario mentioned in Section 7.3.2 are shown in Table 7.2.

Table 7.2: Test Cases for Scenario 2

Test Case ID	Test Case Desc	Req Code	Test Data	Expected Result
TC03	Execute primary transaction workflow	FR02	Valid input payload	Data persisted and status updated
TC04	Handle unexpected edge cases	FR02	Boundary input values	Proper exception handling and feedback

7.4 Non-Functional Testing

7.4.1 Performance and Load Testing

Describe performance benchmarks, response times, throughput, and stress test observations.

7.4.2 Security Testing

Detail vulnerability scans, authentication security, SQL injection / XSS checks, and authorization verification.

7.4.3 Usability Testing

Evaluate user satisfaction, ease of use, and feedback collected from representative end users.

7.5 Requirements Traceability Matrix

Provide a cross reference that traces components and test cases back to the requirements in the SRS chapter. Use a tabular format to show which system components satisfy each functional requirement from Chapter 3, as shown in Table 7.3.

Table 7.3: Requirements Traceability Matrix

Req. ID	Req Description	Module / Class	Test Cases ID	Status
FR01	User authentication	AuthController	TC01, TC02	Verified
FR02	Core transaction logic	TransactionService	TC03, TC04	Verified

7.6 Evaluation and Discussion

Discuss how the test results prove that the system satisfies the original goals and objectives stated in Chapter 1.

7.7 Summary

Short summary of the testing phase and overall quality assessment.

CHAPTER 8

Future Work & Conclusion

This concluding chapter outlines promising avenues for future research and enhancements, followed by a final synthesis of the project outcomes and contributions.

8.1 Future Work

Promising technical enhancements and future extensions identified for subsequent releases of the MedLink platform:

- **Integration with National Healthcare Databases**: Direct synchronization with governmental electronic health record (EHR) systems and health insurance clearinghouses.
- **Advanced Clinical Decision Support Systems (CDSS)**: Expanding the AI engine to evaluate personalized genomic medicine indicators and complex multi-drug therapeutic regimens.
- **Automated IoT Dispensing Integration**: Connecting the platform with automated robotic dispensing cabinets and smart warehouse shelf sensors for automated physical inventory counts.
- **Telepharmacy & Video Consultation**: Integrating real-time secure WebRTC teleconsultation enabling remote pharmacists to consult with isolated or rural patients.

8.2 Conclusion

Comprehensive synthesis of the completed graduation project. Summarizing how the software engineering lifecycle—from literature analysis and formal SRS specifications to architectural design, full-stack implementation, and multi-tier verification—successfully delivered the MedLink Smart Pharmacy Platform, achieving all original goals set forth in Chapter 1.

Bibliography

[1] CORMEN, T. H. *Introduction to algorithms*. MIT press, 2009.

[2] HOOKS, I. Writing good requirements. In *INCOSE International Symposium* (1994), vol. 4, Wiley Online Library, pp. 1247–1253.

APPENDIX A

Git Repository

Link:
Add screenshots from your repository showing your project. Also, add some charts from the insights as shown in Figure A.1

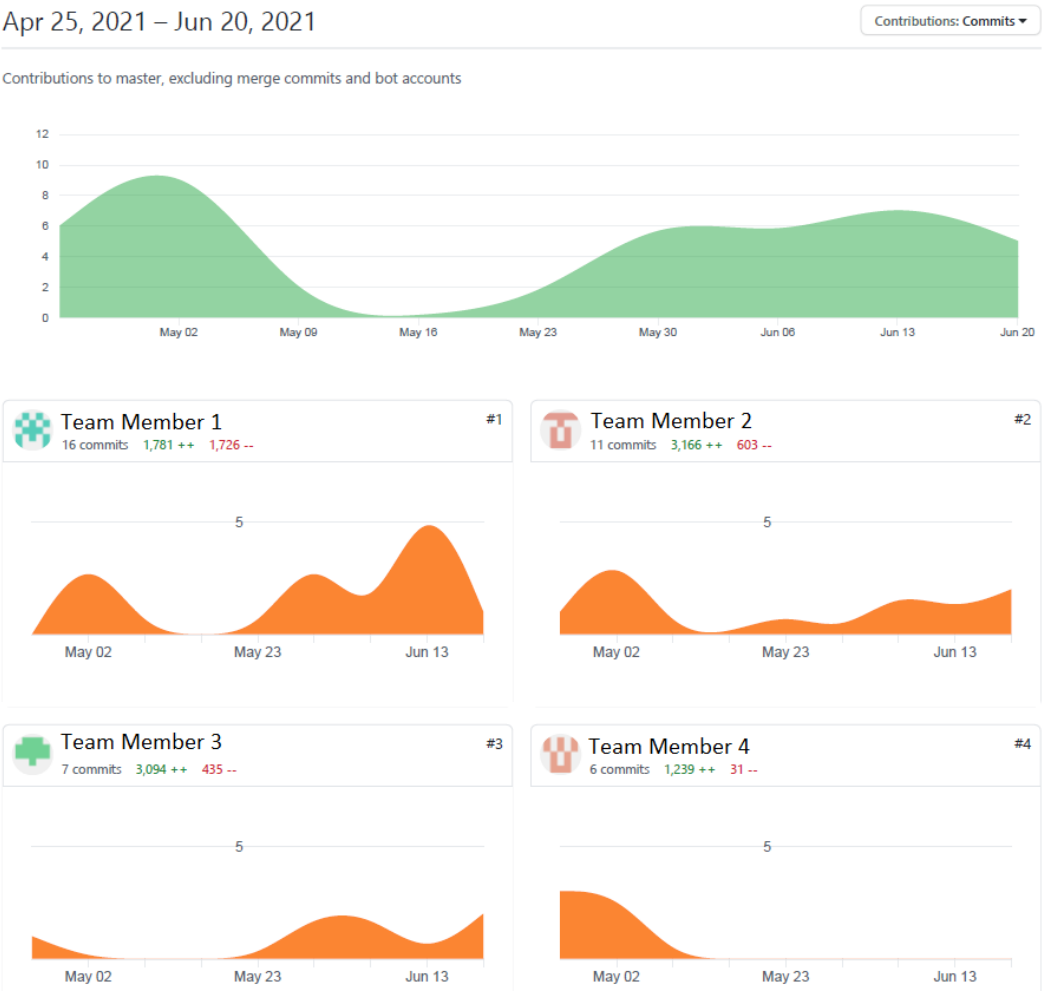


Figure A.1: Git Insights